

AEGISPAY

Production Database Schema

Simple, Secure & Multi-Tenant PostgreSQL Design.

One shared PostgreSQL database for every merchant. Which tables exist, what each stores, and how they connect — in plain words.

1. Database overview

AegisPay uses **one** PostgreSQL database for **many merchants**. Each table that belongs to a merchant carries a **tenant_id**, and PostgreSQL Row-Level Security (RLS) keeps them apart. The schema stays small by reusing tables and JSONB instead of creating new tables.

Group	Tables
Identity & Tenants	tenants · users · tenant_users · agents · agent_sessions
Commerce	products · carts · cart_items · inventory_reservations · orders · order_items
Control & Safety	policies · authorizations · approvals
Payments	payments · refunds · webhook_events
GROW	opportunities · campaigns · campaign_budget_ledger
Platform Reliability	idempotency_keys · outbox_events · audit_events

2. Database architecture

The data is split into a few small groups. The AI layer can propose (catalog, carts, opportunities), but the financial and audit tables are owned by the control plane and payment services.

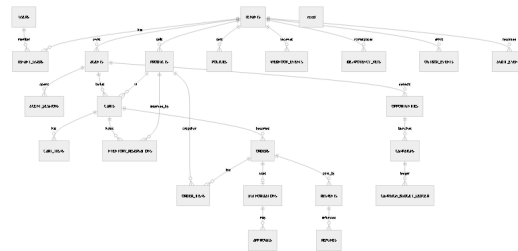


Figure 1 — The complete ER diagram. Only table, primary key and relationships are shown.

3. Complete ER diagram

Every table hangs off a merchant (tenants). The one clean diagram above shows it all at a glance.

4. Identity & Tenants tables

TABLE: tenants

Purpose

One merchant. This is the top of the whole tenant tree.

Columns

Column	Type	Meaning
id	UUID	Merchant ID
slug	TEXT	short unique name
name	TEXT	display name
currency	CHAR(3)	INR
status	TEXT	ACTIVE / SUSPENDED
created_at	TIMESTAMP	when created

Primary Key

id

Foreign Keys

None (top of tree)

Important Indexes

slug (unique)

Used by

TABLE: users

Purpose

A person who can log in. Global identity, shared across merchants.

Columns

Column	Type	Meaning
id	UUID	User ID
email	TEXT	login email (unique)
password_hash	TEXT	never plaintext
status	TEXT	ACTIVE / DISABLED
created_at	TIMESTAMP	when created

Primary Key

id

Foreign Keys

None

Important Indexes

email (unique)

Used by

TABLE: tenant_users

Purpose

Links a user to a merchant with a role. This is the access-control boundary.

Columns

Column	Type	Meaning
id	UUID	Row ID
tenant_id	UUID	merchant
user_id	UUID	person
role	TEXT	admin / ops / approver
created_at	TIMESTAMP	when added

Primary Key

id

Foreign Keys

tenant_id → tenants.id

user_id → users.id

Important Indexes

(tenant_id, role)

(tenant_id, user_id) unique

Used by

TABLE: agents

Purpose

An AI agent's identity, its permissions and allowed tools.

Columns

Column	Type	Meaning
id	UUID	Agent ID
tenant_id	UUID	merchant
type	TEXT	SELL / GROW
scopes	JSONB	what it may do
allowed_tools	JSONB	which tools it may call
trust_level	TEXT	HIGH / MED / LOW
status	TEXT	ACTIVE / SUSPENDED / REVOKED
expires_at	TIMESTAMP	optional expiry

Primary Key

id

Foreign Keys

tenant_id → tenants.id

owner_user → users.id

Important Indexes

(tenant_id, status)

Used by

TABLE: agent_sessions

Purpose

One live session for an agent, so it can be revoked.

Columns

Column	Type	Meaning
id	UUID	Session ID
tenant_id	UUID	merchant
agent_id	UUID	agent
session_key	TEXT	opaque key
device	TEXT	device hint
started_at	TIMESTAMP	when started
ended_at	TIMESTAMP	when ended

Primary Key

id

Foreign Keys

tenant_id → tenants.id

agent_id → agents.id

Important Indexes

(agent_id)

Used by

5. Commerce tables

TABLE: products

Purpose

Catalog. The price lives here and is read-only to the AI.

Columns

Column	Type	Meaning
id	UUID	Product ID
tenant_id	UUID	merchant
sku	TEXT	stock code
name	TEXT	product name
category	TEXT	shoes / food
price_minor	BIGINT	price in paise / kobo
currency	CHAR(3)	INR
status	TEXT	ACTIVE / HIDDEN
agent_safe_summary	TEXT	what AI may display
meta	JSONB	extra attributes

Primary Key

id

Foreign Keys

tenant_id → tenants.id

Important Indexes

(tenant_id, sku) unique

(tenant_id, category, status)

Used by

TABLE: carts

Purpose

A basket. Server-priced and hashed so it can't be silently changed.

Columns

Column	Type	Meaning
id	UUID	Cart ID
tenant_id	UUID	merchant
agent_id	UUID	agent that built it
status	TEXT	ACTIVE / LOCKED / EXPIRED
cart_hash	TEXT	snapshot of contents
price_version	TEXT	price snapshot
total_minor	BIGINT	sum total
expires_at	TIMESTAMP	when it times out

Primary Key

id

Foreign Keys

tenant_id → tenants.id

agent_id → agents.id

customer_id → users.id

Important Indexes

(tenant_id, status)

Used by

TABLE: cart_items

Purpose

The lines inside a cart; each has a server-copied price.

Columns

Column	Type	Meaning
id	UUID	Line ID
tenant_id	UUID	merchant
cart_id	UUID	cart
product_id	UUID	product
quantity	INT	how many
unit_price_minor	BIGINT	price copied from product
line_total_minor	BIGINT	qty × price

Primary Key

id

Foreign Keys

cart_id → carts.id

product_id → products.id

Important Indexes

(cart_id)

Used by

TABLE: inventory_reservations

Purpose

Holds stock during checkout so it can't oversell.

Columns

Column	Type	Meaning
id	UUID	Reservation ID
tenant_id	UUID	merchant
product_id	UUID	product
cart_id	UUID	cart
quantity	INT	reserved qty
reserved_until	TIMESTAMP	release time
status	TEXT	RESERVED / CONSUMED / RELEASED

Primary Key

id

Foreign Keys

product_id → products.id

cart_id → carts.id

Important Indexes

(product_id, status)

Used by

TABLE: orders

Purpose

The frozen purchase. Carries policy version, cart hash and the AI intent snapshot.

Columns

Column	Type	Meaning
id	UUID	Order ID
tenant_id	UUID	merchant
cart_id	UUID	source cart
agent_id	UUID	agent
authorization_id	UUID	permission used
cart_hash	TEXT	cart snapshot
policy_version	TEXT	which policy ran
total_minor	BIGINT	final amount
status	TEXT	order state
intent	JSONB	AI intent snapshot
idempotency_key	TEXT	no duplicate order

Primary Key

id

Foreign Keys

cart_id → carts.id

agent_id → agents.id

authorization_id → authorizations.id

Important Indexes

(tenant_id, status)

(tenant_id, created_at)

Used by

TABLE: order_items

Purpose

Immutable product snapshot per line — proof of the price at purchase time.

Columns

Column	Type	Meaning
id	UUID	Line ID
tenant_id	UUID	merchant
order_id	UUID	order
product_id	UUID	product
product_snapshot	JSONB	name + price frozen
quantity	INT	qty

Column	Type	Meaning
unit_price_minor	BIGINT	price
line_total_minor	BIGINT	total

Primary Key

id

Foreign Keys

order_id → orders.id

product_id → products.id

Important Indexes

(order_id)

Used by

6. Control & Safety tables

TABLE: policies

Purpose

The merchant's deterministic rules. Versioned as new rows (no separate versions table).

Columns

Column	Type	Meaning
id	UUID	Policy ID
tenant_id	UUID	merchant
name	TEXT	policy name
version	INT	version number
status	TEXT	ACTIVE / ARCHIVED
rules	JSONB	the DSL / caps
created_at	TIMESTAMP	when created

Primary Key

id

Foreign Keys

tenant_id → tenants.id

Important Indexes

(tenant_id, name, version) unique

Used by

TABLE: authorizations

Purpose

An explicit permission to pay this exact card — bound, single-use and expiring. Holds the risk decision.

Columns

Column	Type	Meaning
id	UUID	Authorization ID
tenant_id	UUID	merchant
agent_id	UUID	agent
user_id	UUID	person
card_id	UUID	card
card_hash	TEXT	bound card
amount_minor	BIGINT	bound amount
policy_version	TEXT	policy that ran
status	TEXT	VALID / USED / EXPIRED / REVOKED
single_use	BOOL	true
expires_at	TIMESTAMP	when it expires
risk	JSONB	risk score + factors

Primary Key

id

Foreign Keys

tenant_id → tenants.id

agent_id → agents.id

cart_id → carts.id

user_id → users.id

Important Indexes

(tenant_id, status)

Used by

TABLE: approvals

Purpose

A human's yes/no on a risky action. Scoped, expiring, single-use.

Columns

Column	Type	Meaning
id	UUID	Approval ID
tenant_id	UUID	merchant
authorization_id	UUID	what it approves
order_id	UUID	related order
scope_hash	TEXT	the exact approved scope
approver_id	UUID	who decided
decision	TEXT	APPROVE / REJECT
expires_at	TIMESTAMP	deadline
created_at	TIMESTAMP	when requested

Primary Key

id

Foreign Keys

authorization_id → authorizations.id

order_id → orders.id

approver_id → users.id

Important Indexes

(tenant_id, status)

Used by

7. Payments tables

TABLE: payments

Purpose

The complete payment lifecycle. UNKNOWN is a real state.

Columns

Column	Type	Meaning
id	UUID	Payment ID
tenant_id	UUID	merchant
order_id	UUID	related order
authorization_id	UUID	permission used
provider	TEXT	razorpay
provider_order_id	TEXT	Razorpay order
provider_payment_id	TEXT	Razorpay payment
amount_minor	BIGINT	amount
currency	CHAR(3)	INR
status	TEXT	payment state, incl UNKNOWN
idempotency_key	TEXT	duplicate protection
failure_code	TEXT	machine code
failure_reason	TEXT	readable reason

Primary Key

id

Foreign Keys

order_id → orders.id

authorization_id → authorizations.id

tenant_id → tenants.id

Important Indexes

(tenant_id, status)

(tenant_id, created_at)

Used by

TABLE: refunds

Purpose

A controlled refund, capped to what was captured. The AI has no unrestricted refund tool.

Columns

Column	Type	Meaning
id	UUID	Refund ID
tenant_id	UUID	merchant
payment_id	UUID	payment
amount_minor	BIGINT	refund amount
currency	CHAR(3)	INR
reason	TEXT	why
status	TEXT	PENDING / PROCESSED / REFUNDED / FAILED
provider_refund_id	TEXT	Razorpay refund

Column	Type	Meaning
idempotency_key	TEXT	one refund per key

Primary Key

id

Foreign Keys

payment_id → payments.id

tenant_id → tenants.id

Important Indexes

(tenant_id, status)

(tenant_id, idempotency_key) unique

Used by

TABLE: webhook_events

Purpose

Provider events, verified and deduplicated. Raw payload saved.

Columns

Column	Type	Meaning
id	UUID	Event ID
tenant_id	UUID	merchant
provider	TEXT	razorpay
provider_event_id	TEXT	id for dedupe
event_type	TEXT	payment.failed etc
payload	JSONB	raw payload
signature_verified	BOOL	true after check
status	TEXT	RECEIVED / APPLIED / DEDUPED
created_at	TIMESTAMP	when received

Primary Key

id

Foreign Keys

tenant_id → tenants.id

Important Indexes

(provider, provider_event_id) unique

(tenant_id, status)

Used by

8. GROW tables

TABLE: opportunities

Purpose

A revenue idea the AI detected from real purchase affinity.

Columns

Column	Type	Meaning
id	UUID	Opportunity ID
tenant_id	UUID	merchant
agent_id	UUID	growth agent
kind	TEXT	cross_sell / upsell / bundle
anchor_product	UUID	product
target_products	JSONB	suggested products
affinity	NUMERIC	correlation
confidence	NUMERIC	how sure
status	TEXT	OPEN / ACTED

Primary Key

id

Foreign Keys

#tenant_id → tenants.id

#agent_id → agents.id

#anchor_product → products.id

Important Indexes

#(tenant_id, status)

Used by

TABLE: campaigns

Purpose

A budget-capped offer with margin, discount, targeting and A/B config. Budget & spent live here.

Columns

Column	Type	Meaning
id	UUID	Campaign ID
tenant_id	UUID	merchant
agent_id	UUID	growth agent
name	TEXT	campaign name
status	TEXT	PENDING / APPROVED / ACTIVE / PAUSED
budget_minor	BIGINT	max spend
spent_minor	BIGINT	spent so far
discount_pct	NUMERIC	cap
min_margin_pct	NUMERIC	floor
targeting	JSONB	who it targets
ab_config	JSONB	A/B split

Primary Key

id

Foreign Keys

#8226;tenant_id → tenants.id

#8226;agent_id → agents.id

Important Indexes

#8226;(tenant_id, status)

Used by

TABLE: campaign_budget_ledger

Purpose

Every budget move as a row, so spend is atomic and cannot overspend.

Columns

Column	Type	Meaning
id	UUID	Row ID
tenant_id	UUID	merchant
campaign_id	UUID	campaign
amount_minor	BIGINT	amount
kind	TEXT	reserve / consume / release
idempotency_key	TEXT	atomic, no double-spend
created_at	TIMESTAMP	when

Primary Key

id

Foreign Keys

#8226;campaign_id → campaigns.id

#8226;tenant_id → tenants.id

Important Indexes

#8226;(campaign_id)

Used by

9. Platform Reliability tables

TABLE: idempotency_keys

Purpose

Replay cache — a repeated request returns the same result instead of repeating.

Columns

Column	Type	Meaning
id	TEXT(built)	(tenant, scope, key)
tenant_id	UUID	merchant
scope	TEXT	order / payment / refund / webhook / budget
key	TEXT	the idempotency key
request_hash	TEXT	hash of the request
response	JSONB	the stored result
expires_at	TIMESTAMP	when it expires

Primary Key

(tenant_id, scope, key)

Foreign Keys

tenant_id → tenants.id

Important Indexes

(tenant_id, scope, key) primary

Used by

TABLE: outbox_events

Purpose

Event written in the same transaction as the change, so nothing is lost.

Columns

Column	Type	Meaning
id	UUID	Event ID
tenant_id	UUID	merchant
aggregate_type	TEXT	order / payment
aggregate_id	TEXT	which row
event_type	TEXT	payment.succeeded
payload	JSONB	event body
published_at	TIMESTAMP	null until sent

Primary Key

id

Foreign Keys

tenant_id → tenants.id

Important Indexes

(published_at) where null

Used by

TABLE: audit_events

Purpose

Append-only, hash-chained, tamper-evident record of every decision.

Columns

Column	Type	Meaning
id	BIGSERIAL	Row ID
tenant_id	UUID	merchant
event_type	TEXT	approval.approved
actor_type	TEXT	USER / AGENT / SYSTEM / WEBHOOK
actor_id	TEXT	who
correlation_id	TEXT	journey id
transaction_id	TEXT	business id
payload	JSONB	details
prev_hash	TEXT	previous link
event_hash	TEXT	this event's hash

Primary Key

id

Foreign Keys

tenant_id → tenants.id

Important Indexes

(tenant_id, created_at)

(transaction_id)

Used by

10. Shared database security / RLS

Merchant-owned tables carry a **tenant_id**. The application DB role can never bypass RLS, and the frontend never decides the tenant.

Merchant A has **tenant_id** = A, Merchant B has **tenant_id** = B.

PostgreSQL RLS automatically blocks A from reading B's rows — even on a buggy query.

Tenant context is set server-side per request with **SET LOCAL**, never taken from the client.

One role migrates/owns the schema; a separate application role only does DML and cannot bypass RLS.



Figure 2 — Request → authentication → tenant membership → server sets tenant → RLS → database

Frontends can request only what belongs to the authenticated tenant. The database enforces it too.

11. State tables

Simple, readable state transitions — no complicated diagrams.

Payment

State	Next
created	payment_pending
payment_pending	captured
captured	completed
payment_pending	unknown (timeout / no webhook)
unknown	reconciliation → provider truth
unknown	captured / failed / still unknown

Refund

State	Next
pending	processing
processing	refunded

Authorization

State	Next
pending	approved
approved	consumed
pending	rejected / expired

Cart

State	Next
active	locked
locked	expired
locked	converted_to_order

If the provider is slow, the payment is unknown — we never retry it blindly. We reconcile (ask the provider), then finish or fail safely.

12. SELL database flow

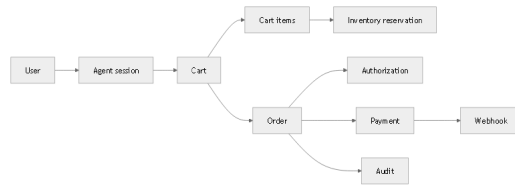


Figure 3 — How a purchase moves through the database

13. GROW database flow

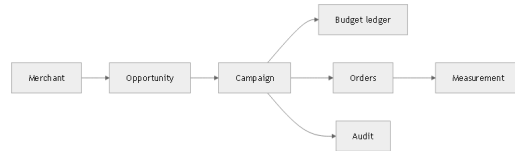


Figure 4 — How a campaign starts, spends and is measured

14. Audit + Transaction Passport

There is **no** passport table. The Transaction Passport is **generated** from the order, order items, authorization, policy, approval, payment and audit events. The audit trail is append-only and hash-chained — it is **tamper-evident**, not tamper-proof.

Source of truth

Thing	Where it lives
Product price	products.price_minor
Cart price	cart_items.unit_price_minor
Order price	orders.total_minor
Payment status	payments.status
Provider truth	Razorpay + verified webhook/reconciliation
Policy	policies
Authorization	authorizations
Audit	audit_events
Cache	Redis — never the source of truth

15. Idempotency + Outbox

Idempotency. Every financial command carries a key; a repeated request returns the stored result.

Designed to prevent duplicate financial effects, not 'impossible'.

Transactional outbox. The business change and its event are written in one transaction, so nothing is lost; a worker publishes to SQS.

16. Why we kept the schema simple

We intentionally did **not** create separate tables for:

Transaction Passport · Risk Decision · AI Intent · Policy Version · Price History · Agent Permissions · Campaign Variants

Reason: these are safely represented using existing tables, immutable snapshots (order_items.product_snapshot), JSONB (risk, intent, scopes, ab_config, rules) or derived views — at this scale. It keeps AegisPay easier to understand and maintain.

17. Final production checklist

Every merchant table has tenant_id + RLS; app role can't bypass it.

Money is an exact whole number (BIGINT) + a currency; no floats.

All financial commands are idempotent.

Payment UNKNOWN is handled and reconciled; no blind retry.

Webhooks are verified and deduplicated.

Outbox + tamper-evident audit are in place.

Authorization is scoped, expiring and single-use.

Refunds are capped and controlled; no unrestricted AI refund.

Secrets are in AWS Secrets Manager, never in the database.